

# Hybrid Inlining: A Framework for Compositional and Context-Sensitive Static Analysis

Jiangchao Liu\*  
Ant Group, Hangzhou, China  
ByteDance, Hangzhou, China

Jierui Liu  
Peng Di  
Diyu Wu  
Hengjie Zheng  
Alex X. Liu  
Ant Group, Hangzhou, China

Jingling Xue  
UNSW Sydney, Australia

## ABSTRACT

Context-sensitivity is essential for achieving good precision in inter-procedural static analysis. To be context-sensitive, top-down analysis needs to fully inline all the statements in a callee at all its callsites, leading to statement explosion. Compositional analysis, which inlines summaries of all the callees, scales up but often loses precision, as it is not strictly context-sensitive. We propose a compositional and strictly context-sensitive framework for static analysis. This framework is based on a key observation: a compositional analysis often loses precision only on some *critical* statements that need to be analyzed context-sensitively. Our approach hybridly inlines the critical statements and the summaries of non-critical statements of each callee, thus avoiding re-analyzing non-critical ones. In addition, our analysis lazily summarizes the critical statements, by stopping propagating the critical statements once the calling context accumulated is adequate. We have designed and implemented several analyses (including a pointer analysis) based on this framework. Our evaluation on the pointer analysis shows that it can analyze large Java programs from the DaCapo benchmark suite and industry in minutes. Compared to context-insensitive analysis, Hybrid Inlining introduces only 65% and 1% additional time overheads on DaCapo and industrial applications, respectively.

## CCS CONCEPTS

• Software and its engineering → Automated static analysis.

## KEYWORDS

context sensitivity, compositional static analysis, pointer analysis

### ACM Reference Format:

Jiangchao Liu, Jierui Liu, Peng Di, Diyu Wu, Hengjie Zheng, Alex X. Liu, and Jingling Xue. 2023. Hybrid Inlining: A Framework for Compositional and Context-Sensitive Static Analysis. In *Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA '23)*, July 17–21, 2023, Seattle, WA, USA.

\*This research was conducted in Ant Group. The author currently works at ByteDance (liujiangchao@bytedance.com).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from [permissions@acm.org](mailto:permissions@acm.org).

ISSTA '23, July 17–21, 2023, Seattle, WA, USA

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0221-1/23/07...\$15.00

<https://doi.org/10.1145/3597926.3598042>

17–21, 2023, Seattle, WA, USA. ACM, New York, NY, USA, 13 pages. <https://doi.org/10.1145/3597926.3598042>

## 1 INTRODUCTION

Context-sensitivity is essential for achieving highly useful precision in inter-procedural static analysis. In general, there are two ways to conduct inter-procedural analysis: top-down and compositional (i.e., bottom-up). Both fall short in terms of realizing the full potential for context-sensitivity in analyzing software at industrial scale.

To enable context-sensitivity in a top-down analysis, the statements of called procedures need to be inlined explicitly or implicitly at each callsite [33, 44]. However, inlining all procedures in a large industrial application often leads to a significant increase in statements, posing practical challenges. Limiting context-sensitivity, such as using k-callsite sensitivity [44] or k-object sensitivity [33], is a potential compromise but can result in precision loss [32]. Despite recent advancements in techniques like parallelization [1], demand-driven analysis [2, 20, 41, 48], selective context-sensitivity [23, 27, 29], and incremental analysis [30, 40], achieving scalability with a high degree of context-sensitivity remains challenging.

A compositional approach [11] starts from the leaf procedures of the call graph, and inlines the summaries of the called procedures at each callsite. Since each procedure is analyzed only once to obtain its summary, it is much easier for the analysis to scale up. However, the summaries are computed with all possible input states (i.e.,  $\top$ ) as a precondition. Inlining such summaries often leads to less precise results than inlining statements. This makes compositional analysis non-strictly context-sensitive. Some works tackle context-sensitivity with subtle designs of summaries [6, 13, 15, 24, 34]. However, these works either are specific to a kind of properties or introduce disjunction that brings potential disjunctive explosion.

In this paper, we propose a compositional and context-sensitive static analysis framework called *Hybrid Inlining*. Our key observation is that the summarization process in a compositional analysis is imprecise only on some statements due to the absence of contexts, referred to as *critical* statements. In other words, the summarization of non-critical statements with/without contexts generates the same postcondition for any precondition. At each callsite, our framework hybridly inlines the critical statements (for context-sensitivity) and the summaries of non-critical statements (for performance) of the called procedures. Thus, the critical statements are propagated from callees to the contexts in their callers. The propagation of a critical statement stops at a callsite where the context is *adequate* (i.e., no further precision improvement with more propagation). Then the

critical statement is summarized. Its adequate context condition enables avoiding unnecessary propagation of critical statements to the root procedures. Compositional analysis with Hybrid Inlining is strictly context-sensitive. Compared with conventional compositional context-insensitive analysis, the non-critical statements are still analyzed once, but the critical statements are analyzed multiple times, depending on the distinct contexts encountered.

Unlike existing compositional analyses [6, 13, 15, 24, 34], the novelty of our approach lies in determining *when* to summarize the statements in a program rather than *how* to summarize them. Another key characteristic of our approach is that it never creates disjuncts, which makes it fundamentally different from all techniques on *when* [57] or *how* [25] to split/merge disjuncts.

Our implementation includes various complex compositional analyses, such as a Java pointer analysis, within Hybrid Inlining. By combining the scalability benefits of compositional analysis with context-sensitivity, we achieve efficient and accurate analyses. This enables us to analyze large programs from the DaCapo benchmark and industrial applications in a matter of minutes [4].

In this paper, we make the following three main contributions:

- We propose a Hybrid Inlining framework for performing compositional analysis, which realizes context-sensitivity at only a small cost on performance.
- We instantiate Hybrid Inlining with several non-trivial analyses including a context-sensitive pointer analysis.
- We share our experience in applying the pointer analysis to 10 open-source benchmarks from DaCapo and 10 real industrial large applications. With Hybrid Inlining, our context-sensitive pointer analysis can analyze large programs as efficiently as the context-insensitive pointer analysis.

## 2 OVERVIEW

In this section, we illustrate the key idea of our Hybrid Inlining framework by applying pointer analysis to a Java program in Figure 1. With this motivating example, we first consider a conventional compositional flow-insensitive pointer analysis, which is context-insensitive. Then we demonstrate how Hybrid Inlining improves its precision with strict context-sensitivity.

In this example, an interface *X* (line 2) declares an abstract procedure *poly()*. Two classes *Y* and *Z* both implement *X* and override the abstract procedure *poly()* differently. For clarity, we use *polyY()* and *polyZ()* to denote the procedure *poly()* in *Y* and *Z*, respectively, where *polyY* is an identity procedure and *polyZ* just returns a new instance of class *Obj*. In class *FacadeImpl*, an identity procedure *id()* at line 18 is defined. The procedure *foo()* calls the abstract procedure *poly()* (at line 25), whose implementation is only known when its first parameter is decided (which means that context-sensitive analysis is needed). The procedures *bar1()* and *bar2()* call *foo()* indirectly with the receiver of the procedure *poly()* specialized as instances of *Y* and *Z*, respectively. Despite the complexity in their code, *bar1()* can actually be seen as an identity procedure and *bar2()* as returning a new instance of class *Obj*. The procedure *service()* is the root procedure. Note that the two assertions in this root procedure are always *true*. However, as mentioned above, these need to be verified by a context-sensitive static analysis.

```

1  public class Obj{}
2  public interface X{
3      Obj poly(Obj obj);
4  }
5  public class Y implements X{
6      @Override
7      public Obj poly(Obj obj){
8          return obj;
9      }
10 }
11 public class Z implements X{
12     @Override
13     public Obj poly(Obj obj){
14         return new Obj();
15     }
16 }
17 public class FacadeImpl{
18     public X id(X x){
19         X tv = x;
20         return tv;
21     }
22     public Obj foo(X x, Obj obj){
23         ...
24         X tx = id(x);
25         return tx.poly(obj);
26     }
27     public Obj mid(X x, Obj obj){
28         return foo(x, obj);
29     }
30     public Obj bar1(Obj obj){
31         return mid(new Y(), obj);
32     }
33     public Obj bar2(Obj obj){
34         return mid(new Z(), obj);
35     }
36     public void service(){
37         Obj first = new Obj();
38         Obj second = bar1(first);
39         Obj third = bar2(first);
40         assert(first == second);
41         assert(first != third);
42     }
43 }

```

Figure 1: An illustrating example.

### 2.1 A Compositional and Context-Insensitive Pointer Analysis

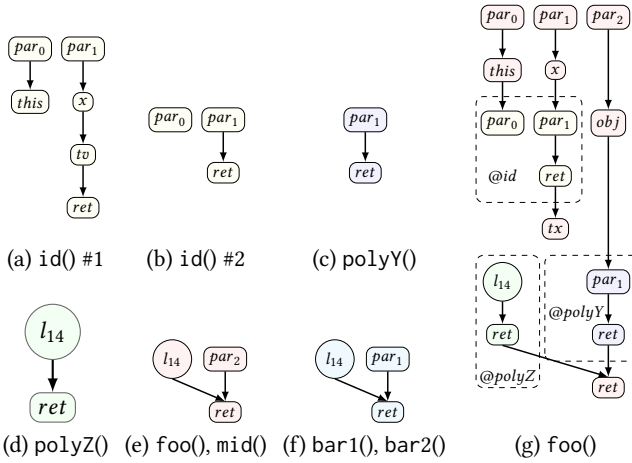
We first define a conventional compositional pointer analysis, which summarizes the procedure *foo()* context-insensitively. It inlines both *polyY()* and *polyZ()* at the callsite at line 25, since the type of the receiver is unknown. This imprecision will cause the verification of the two assertions in the procedure *service()* to fail. We elaborate on this particular pointer analysis below.

**Abstract states.** An abstract state is a computer representable abstraction of a set of concrete program states. In this pointer analysis, we define an abstract state  $\alpha \in \mathfrak{A}$  as a set of set constraints on program variables (denoted as  $v \in \mathbb{V}$ ) and locations (i.e., allocation sites, denoted as  $l \in \mathbb{L}$ ) of the form  $v \supseteq v'$  and  $v \supseteq \{l\}$ . For instance,  $tv \supseteq x$  (from line 19) means that the set of locations pointed to by *tv* is a superset of those pointed to by *x*. The constraint  $first \supseteq \{l_{37}\}$  (from line 37) indicates that the variable *first* may point to the allocation site declared at line 37. By convention, there is a special abstract state  $\top$  that represents all possible concrete states.

**Abstract summaries.** A compositional analysis is built on abstract summaries  $\epsilon \in \mathfrak{A} \times \mathfrak{A}$ , which are relations on abstract states. It maps a given precondition to a postcondition (both represented by abstract states). A compositional analysis can summarize any program into an abstract summary, achieved with summarization.

**Summarization.** We introduce additional symbolic variables to denote the return value and the formal parameters of each procedure. For instance, the body of the procedure *id()* at lines 18–22 is rewritten to “*this*=*par*<sub>0</sub>@*id*; *X* *x* = *par*<sub>1</sub>@*id*; *X* *tv* = *x*; *ret*@*id* = *tv*”, where *par*<sub>0</sub>@*id* denotes its 0th parameter and *ret*@*id* denotes its return value. The summarization on these assignment statements is standard. On the assignment *X* *tv* = *x* at line 19, the analysis generates the summary  $\forall \alpha \in \mathfrak{A}, \alpha \mapsto \alpha \cup \{tv \supseteq x\}$ , which indicates that for any precondition, the statement at line 19 just adds a new constraint  $tv \supseteq x$ . The summary of *id()* is obtained after applying summarization to the four statements as  $\forall \alpha \in \mathfrak{A}, \alpha \mapsto \alpha \cup \{ret@id \supseteq tv, tv \supseteq x, x \supseteq par_1@id, this \supseteq par_0@id\}$ . Actually, all abstract summaries are of the form  $\forall \alpha \in \mathfrak{A}, \alpha \mapsto \alpha \cup \alpha'$ , which

means that each summary just adds new constraints  $\alpha'$  to any precondition. In the following, we will only show the new constraints in each summary for simplicity. We also denote such constraints with a graphical representation as shown in Figure 2(a) where “ $\leftarrow$ ” represents “ $\supseteq$ ”. In addition, we omit the label  $@id$  if the procedure name is obvious. In each figure, colors are used to distinguish variables from different procedures. In these constraints, some variables like  $this$ ,  $tv$  and  $x$  cannot be accessed outside the procedure  $id()$ . It is not necessary to include them in the summary. Thus a transitive closure is computed and the constraints on the three variables are safely removed. The resulting summary is  $\{ret@id \supseteq par_1@id\}$  as shown in Figure 2(b), which indicates an identity procedure. Following the same principle, the summary of  $polyY()$  (resp.  $polyZ()$ ) from class  $Y$  (resp.  $Z$ ) is found as shown in Figure 2(c) (resp. Figure 2(d)). In these figures,  $l_{14}$  denotes the allocation site at line 14. We also omit some irrelevant nodes like  $par_0@polyY$ .



**Figure 2: Procedure summaries generated by a compositional and context-insensitive pointer analysis.**

In this compositional and context-insensitive analysis, the summaries of the called procedures are inlined at each callsite. For instance, on the procedure invocation at line 24 (i.e.,  $x \text{ tx} = id(x)$ ), the analysis inlines the summary of  $id()$ . The result is shown in Figure 2(g). The analysis loses no precision for this particular inlining. However, it will lose precision on the following statement  $ret@foo = tx.poly(obj)$  at line 25 due to context-insensitivity.

In the statement  $ret@foo = tx.poly(obj)$ , the implementation of the abstract procedure  $poly()$  is only known when the points-to set of its receiver (i.e.,  $tx$ ) is decided. The analysis can infer that this points-to set depends on the first parameter of the procedure (i.e.,  $tx \supseteq par_1@foo$  as shown in Figure 2(g)). This means that the choice on the implementation of  $poly()$  depends on the context, which is absent at this stage. The compositional analysis has to assume all possible input states, where  $par_1@foo$  can point to any allocation site. Thus, the analysis has to enumerate all implementations of  $poly()$  on this procedure invocation<sup>1</sup>. Thus, the summaries of  $polyY()$  in Figure 2(c) and  $polyZ()$  in Figure 2(d) are both inlined,

<sup>1</sup>we do not take the error state into account for simplicity

as shown in Figure 2(g). This brings precision loss. Note that we omit some irrelevant constraints like  $par_0@polyY \supseteq tx$ .

The summary of  $foo()$  is obtained as in Figure 2(e) after removing the inaccessible variables (line 23 is assumed to contain irrelevant code). The connection between the context and the choice on the implementation of  $poly()$  is lost. Similarly, the summary of  $mid()$  is also obtained as in Figure 2(e). Inlining this summary at the callsite at line 31 (i.e.  $mid(new Y(), obj)$ ) will produce the summary of  $bar1()$  in Figure 2(f), which imprecisely includes  $l_{14}$  in the return value. Following the same principle, the summary of  $bar2()$  is also obtained as Figure 2(f). They are less precise than inlining the statements of  $foo()$  and  $mid()$  at each callsite. Due to this imprecision, the analysis fails to verify either of the two assertions in  $service()$ . Worse still, this pointer analysis also generates a call graph that imprecisely includes the unreachable call  $string\ bar1.foo.polyZ$ .

One way to achieve context-sensitivity on conventional compositional analysis is to use disjunctive summaries. However, disjunction is known to be hard to manage and the size of which can grow rapidly [25, 57]. We will illustrate this later in this section.

## 2.2 Hybrid Inlining

Now we show how to generate a compositional and context-sensitive pointer analysis with Hybrid Inlining proposed in this paper. The main idea is based on the observation that only the summarization of the virtual invocation at line 25 needs context-sensitivity. Thus, our approach will postpone the summarization of that statement by embedding the statement itself into the summary of other statements in  $foo()$ . Such a critical statement is propagated along the call chain until  $bar1()$  and then summarized, where the type of its receiver is determined. This process brings context-sensitivity without introducing disjunction, as formalized later.

**Hybrid summary.** We use  $stmt \in \mathbb{S}$  to denote program statements. We define a *hybrid summary* as  $(c, S)$ , where  $S \in 2^{\mathbb{S}}$  denotes a set of critical statements to be propagated and  $c \in \mathcal{V} \times \mathcal{V}$  is an abstract summary from the aforementioned compositional pointer analysis. A formal definition of critical statements will be given in Section 3. Here, we just give an intuitive introduction with the code in Figure 1. The statement  $x \text{ tv} = x$  at line 19 is non-critical because no matter what the precondition is, the postcondition is always  $tv \supseteq x$ . However, the virtual invocation statement  $tx.poly(obj)$  at line 25 is critical because different preconditions can generate different postconditions (causing different procedures to be dispatched).

**Summarization and inlining.** Our approach produces one summary for  $id()$ , the same as in Figure 2(b) and inlines it at line 24 (i.e.,  $x \text{ tx} = id(x)$ ). However, on summarizing  $foo()$ , the situation is different. Our approach finds that summarizing the critical virtual invocation statement  $tx.poly(obj)$  without any context will cause precision loss, and will thus add it to  $S$  instead of summarizing it immediately. The resulting hybrid summary of  $foo()$  is  $(\{tx \supseteq par_1@foo, obj \supseteq par_2@foo, ret \supseteq ret@poly\}, \{tx.poly(obj)\})$ , as shown in Figure 3(a). The thick box denotes the critical statement and is connected with all variables that it accesses. In our approach, critical statements are just like placeholders in the hybrid summary and will be summarized when their contexts are ready.

By inlining the hybrid summary of  $foo()$  and removing inaccessible variables, we obtain the hybrid summary of  $mid()$  (Figure 3(b)).

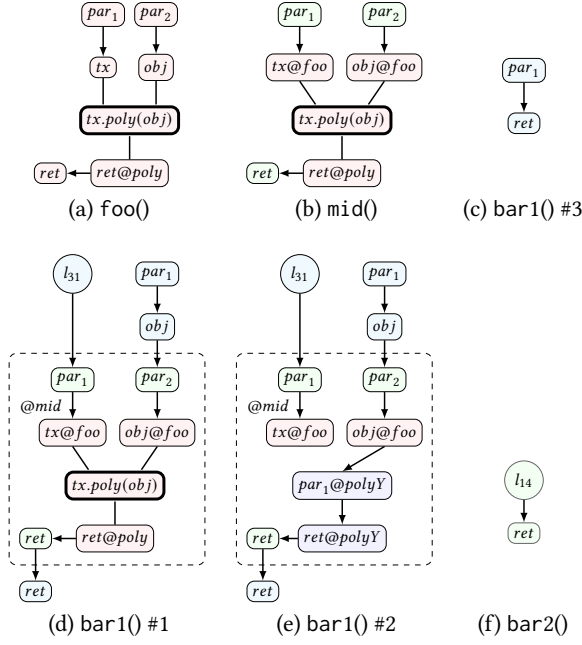


Figure 3: Procedure summaries via Hybrid Inlining.

Inlining this hybrid summary at line 31 in bar1() results in its hybrid summary (Figure 3(d)). The critical statement `tx.poly(obj)` is propagated from `foo()` to `bar1()`.

**Context-ready predicates.** We introduce a *context-ready* predicate  $\Phi$ , which decides whether a critical statement should be summarized with the current context. A simple way to define it is to summarize critical statements that propagate  $k$  steps, i.e., callsites, giving rise to  $k$ -callsite-sensitivity. But we prefer to have a better definition for  $\Phi$ . In Figure 3(d), the receiver of the virtual invocation statement `tx.poly(obj)` is known to point to  $l_{31}$  (i.e. the allocation of an instance of  $Y$  at line 31). Our analysis also infers that the points-to set of the receiver `tx` will not change even the analysis propagates this virtual invocation statement further, because `tx` and all its aliased variables are not accessible out of the scope of `bar1()`. This context is considered as an *adequate* context. We define  $\Phi$  as checking whether the context is adequate. Thus, the propagation of `tx.poly(obj)` stops and our analysis summarizes it with the current context. In this case, our analysis dispatches the virtual invocation to the implementation of `poly()` in  $Y$  (i.e., `polyY()`). Then our analysis inlines the summary of `polyY()` as shown in Figure 3(e). After removing the inaccessible variables, the summary of the procedure `bar1()` is generated as shown in Figure 3(c). Similarly, the summary of `bar2()` is computed as shown in Figure 3(f).

With the summaries of `bar1()` and `bar2()` computed this way by our analysis, the two assertions at lines 40 and 41 can be verified successfully. In addition, the unreachable call string `bar1.foo.polyZ` that is introduced by the conventional compositional analysis has been ruled out in our call graph thus precisely built.

**Comparison with conventional compositional approaches.** Compared with the conventional compositional context-insensitive analysis, Hybrid Inlining achieves context-sensitivity at some additional performance overhead on analyzing the critical statement

`tx.foo(poly)` twice in this example. This is a very good trade-off, especially if most statements in a program are non-critical. Fortunately, this is the case as validated in Section 5.

Another way to make compositional analysis context-sensitive is to resort to disjunction. For example, multiple summaries and disjunctive abstract states [13, 15, 57] fall into this category. With this approach, `tx.poly(obj)` is summarized as:

$$\begin{aligned} \text{if } \{tx \supseteq \{l_Y\}\} \in \mathbf{a}, \mathbf{a} \mapsto \mathbf{a} \cup \{\text{ret@foo} \supseteq \text{par}_1\text{@foo}\} \\ \text{if } \{tx \supseteq \{l_Z\}\} \in \mathbf{a}, \mathbf{a} \mapsto \mathbf{a} \cup \{\text{ret@foo} \supseteq \{l_{14}\}\} \end{aligned}$$

where  $l_Y$  (resp.  $l_Z$ ) denotes an allocation site of object  $Y$  (resp.  $Z$ ). This summary can achieve the same precision as Hybrid Inlining. However, its main limitation [25, 57] is well-known. The size of disjuncts increases exponentially in terms of the number of virtual invocation statements. Suppose that class  $X$  has a total of 10 different implementations and the procedure `foo()` is added one parameter that is used as the receiver of another virtual invocation of `poly()`. Then such a traditional approach will end up building as many as 100 disjuncts in order to be equivalently precise as Hybrid Inlining. It takes a lot of time and memory to generate and maintain such disjuncts. However, Hybrid Inlining just needs to propagate only two critical statements in this case. Sometimes, it can be just impossible to generate disjunction, because the number of possibilities is unbounded. Section 4.1 gives such an example.

**Comparison with conventional top-down approaches.** The two assertions in Figure 1 can be non-trivial for top-down Andersen-style pointer analysis to verify. It needs at least a 3-callsite-sensitive analysis. Such an analysis has to analyze each of the procedures `foo()`, `id()` and `mid()` twice, bringing significant performance loss. In contrast, our approach just needs to analyze the statement at line 25 twice and summarize all the other statements once only. Selective context-sensitivity [49] for top-down analysis does not help for this example since all procedures need context-sensitivity.

Some CFL-based inter-procedural analyses [37, 38, 46] are top-down but also utilize summaries to boost performance. Actually, the main idea behind the compositional analysis in Section 2.1 is similar to the summarization process in [37]. However, those analyses do not delay the summarization of critical statements and thus cannot avoid generating multiple summaries for the same procedure (like `mid()` and `foo()`). This can again bring serious performance loss.

### 3 METHODOLOGY

In this section, we present the main idea behind Hybrid Inlining in flow-insensitive analysis. A similar approach can be applied to flow-sensitive analysis. For brevity, we omit complex formulas related to soundness, although our framework ensures soundness.

#### 3.1 Compositional Flow-Insensitive Analysis

We first formalize a general compositional analysis and then describe how to make it context-sensitive with Hybrid Inlining.

To define compositional analysis, we first introduce abstract states  $\mathbf{a} \in \mathfrak{A}$  used to describe program properties. For instance, the abstract states defined in Section 2 utilize set constraints to describe points-to relations of program variables. Such abstract states form a lattice, where  $\top$  and  $\perp$  denote *top* and *bottom*, respectively.



Compositional analysis is built on abstract summaries  $c \in \mathbb{C}$  :  $\mathbb{A} \Rightarrow \mathbb{A}$ , which are relations on abstract states mapping preconditions to postconditions. A conventional compositional analysis is characterized by the following summaries and operators:

- An abstract transfer function  $\text{trans} : (\mathbb{A} \times \mathbb{S}) \Rightarrow \mathbb{C}$  that summarizes a statement in  $\mathbb{S}$  to a summary in  $\mathbb{C}$  with a context in  $\mathbb{A}$ . A context represents a condition that must be satisfied by the precondition; and
- A join operator  $\sqcup : (\mathbb{A} \times \mathbb{C} \times \mathbb{C}) \Rightarrow \mathbb{C}$  that over-approximates the effect of the union of two summaries with a context in  $\mathbb{A}$ .

A flow-insensitive analysis sees a procedure as a set of statements. A compositional and flow-insensitive analysis summarizes each statement in a program with  $\text{trans}$  and join the results with  $\sqcup$ . With the function  $\Lambda$  that maps a program  $P$  to the set of statements  $\mathcal{S}$  contained in  $P$ , we define a summarization function  $\text{sum}$  that maps a program to a summary with a given context  $\alpha$ :

$$\text{sum}(\alpha, P) := \bigsqcup_{\text{stmt} \in \Lambda(P)} \text{trans}(\alpha, \text{stmt})$$

where  $\sqcup_\alpha$  denotes the join operator with  $\alpha$  as the context.

The abstract semantics  $\llbracket \cdot \rrbracket$  of this compositional analysis is:

$$\llbracket P \rrbracket(\alpha) := \text{sum}(\top, P)(\alpha)$$

A compositional analysis first summarizes a program with  $\top$  as the context, producing a summary  $c$ , and then applies it to any precondition  $\alpha$  to compute the postcondition  $c(\alpha)$ .

We also use  $\Lambda$  to map a procedure  $\text{proc}$  to its statements. The transfer function for procedure invocation is:

$$\text{trans}(\alpha, \text{call proc}) := \text{sum}(\alpha, \Lambda(\text{proc}))$$

If we take  $\top$  as the context, compositional analysis just needs to compute the summary  $\text{sum}(\top, \Lambda(\text{proc}))$  of each procedure once and for all. This ensures that its analysis time increases almost linearly as the size of the program being analyzed. This represents a significant performance advantage over top-down analysis.

This is a standard abstract-relation-based compositional analysis.

*Example 3.1.* The Andersen-style pointer analysis discussed in Section 2.1 falls into the category of compositional, flow-insensitive and context-insensitive analyses defined in this section.

However, despite its performance advantage, compositional analysis is much less explored than top-down analysis in practice. This is because it is difficult to design a precise compositional analysis, with respect to strict context-sensitivity, in practice.

For a compositional analysis, strict context-sensitivity means:

$$\forall \alpha \in \mathbb{A}, \text{sum}(\top, \Lambda(\text{proc}))(\alpha) = \text{sum}(\alpha, \Lambda(\text{proc}))(\alpha)$$

Hence, for any precondition, applying the abstract summary of a procedure yields the same postcondition as re-analyzing its body with that precondition. To ensure this, the transfer functions  $\text{trans}$  and the join operator  $\sqcup$  must preserve precision:

$$\mathcal{P}_{\text{trans}}(\alpha, \text{stmt}) := \forall \alpha' \in \mathbb{C}, \text{trans}(\alpha, \text{stmt})(\alpha') = \text{trans}(\alpha', \text{stmt})(\alpha')$$

$$\mathcal{P}_{\sqcup}(\alpha, c_0, c_1) := \forall \alpha' \in \mathbb{A}, (c_0 \sqcup_\alpha c_1)(\alpha') = (c_0 \sqcup_{\alpha'} c_1)(\alpha')$$

A compositional analysis is context-sensitive, if for any statement  $\text{stmt}$  and abstract summaries  $c_0, c_1 \in \mathbb{C}$ , both  $\mathcal{P}_{\text{trans}}(\top, \text{stmt})$  and  $\mathcal{P}_{\sqcup}(\top, c_0, c_1)$  hold, which indicate that, the transfer function

and join operator lose no precision with  $\top$  as the context. There are two main difficulties in designing such a compositional analysis: (1) a domain of computer representable abstract summaries (i.e., sets of symbolic relations) is hard to design, and (2) the behaviors of statements can be so complex that summarizing them without contexts can hardly keep all behaviors in the resulting summary.

Due to these two difficulties (especially the second one), conventional compositional analysis often gives up strict context-sensitivity, or as in many prior works [13, 15, 57], addresses the second difficulty with disjunctive summaries (leading to disjunctive explosion).

### 3.2 Hybrid Inlining

In this work, we propose to make conventional compositional analysis strictly context-sensitive without introducing disjunction. Our approach is centered on one key observation: given a conventional compositional analysis, the predicate  $\mathcal{P}_{\text{trans}}(\top, \text{stmt})$  evaluates to *False* only on some statements, which are called *critical* statements.

A statement  $\text{stmt}$  is *critical* if  $\mathcal{P}_{\text{trans}}(\top, \text{stmt})$  does not hold:

$$\exists \alpha \in \mathbb{C}, \text{trans}(\top, \text{stmt})(\alpha) \supset \text{trans}(\alpha, \text{stmt})(\alpha)$$

Similarly, a statement is *non-critical* if  $\mathcal{P}_{\text{trans}}(\top, \text{stmt})$  holds.

$$\forall \alpha \in \mathbb{C}, \text{trans}(\top, \text{stmt})(\alpha) = \text{trans}(\alpha, \text{stmt})(\alpha)$$

Therefore, we can see that, a statement is non-critical when summarizing it with  $\top$  as the context is as precise as summarizing it with any context. Otherwise, this statement is critical.

Based on this observation, we propose a hybrid approach for the treatment of critical and non-critical statements during the summarization process. Instead of using top-down analysis, which includes all statements of a callee, or compositional analysis, which includes the summaries of all statements of a callee, our approach incorporates all critical statements and the summaries of non-critical statements in a callee. We achieve this with a *hybrid summary*  $\mathfrak{h}$ , which is defined as a pair consisting of an abstract summary  $c \in \mathbb{C}$  and a set of critical statements  $\mathcal{S} \subseteq 2^{\mathbb{S}}$  to be propagated:

$$\mathfrak{h} := (c, \mathcal{S})$$

We define the join operator  $\sqcup^h$  for hybrid summaries as follows:

$$(c_0, \mathcal{S}_0) \sqcup^h (c_1, \mathcal{S}_1) := (c_0 \sqcup c_1, \mathcal{S}_0 \cup \mathcal{S}_1)$$

A hybrid summary is computed with a function  $\text{sum}^h$  that summarizes any program  $P$  to a hybrid summary  $\mathfrak{h}$ . On atomic statements,  $\text{sum}^h$  just summarizes them with  $\text{trans}$  if they are non-critical and keep them as they are if they are critical:

$$\text{sum}^h(\text{stmt}) := \begin{cases} (\text{trans}(\top, \text{stmt}), \emptyset), & \text{if } \mathcal{P}_{\text{trans}}(\top, \text{stmt}) \\ (\perp, \{\text{stmt}\}), & \text{otherwise.} \end{cases}$$

For a program  $P$ ,  $\text{sum}^h$  just keeps applying the join operator  $\sqcup^h$  to the summarization of each statement in  $\Lambda(P)$ :

$$\text{sum}^h(P) := \bigsqcup_{\text{stmt} \in \Lambda(P)}^h \text{sum}^h(\text{stmt})$$

Now we define the abstract semantics with Hybrid Inlining. A hybrid summary contains both statements and abstract summaries. Thus, we rely on an operator **apply**, which summarizes the statements and applies the abstract summaries to compute the postcondition with respect to a given precondition:

$$\text{apply}((c, \mathcal{S}), \alpha) := (c \sqcup_\alpha \text{sum}(\alpha, \mathcal{S}))(\alpha)$$

The abstract semantics of Hybrid Inlining on flow-insensitive analysis is defined as follows:

$$\llbracket P \rrbracket^h(\alpha) := \text{apply}(\text{sum}^h(P), \alpha)$$

**THEOREM 3.2.** *The abstract semantics of Hybrid Inlining on flow-insensitive analysis is strictly context-sensitive if  $\mathcal{P}_{\sqcup}(\top, \alpha_0, \alpha_1)$  holds for any  $\alpha_0, \alpha_1 \in \mathcal{C}$*

**3.2.1 Adequate Contexts.** In the aforementioned formalization, critical statements are propagated to the root procedure. However, this is not always necessary. A critical statement can become non-critical during its propagation. In the example from Section 2, the virtual invocation statement  $tx.poly(obj)$  is no longer critical any more when it is propagated to  $bar1()$ . This indicates that the context at  $bar1()$  becomes *adequate* (to stop any further propagation).

Given a hybrid summary  $(\mathcal{C}, S)$ , the context for a critical statement  $\text{stmt} \in S$  is  $\text{apply}((\mathcal{C}, S), \top)$ , which is more precise than  $\top$ . Thus it is possible that, for some statements,  $\mathcal{P}_{\text{trans}}(\top, \text{stmt})$  does not hold, but  $\mathcal{P}_{\text{trans}}(\text{apply}((\mathcal{C}, S), \top), \text{stmt})$  holds. This means that some critical statements can become non-critical during their propagation. In this case, the context  $\text{apply}((\mathcal{C}, S), \top)$  is adequate for these critical statements. We can stop their propagation and safely summarize them at that point.

Let  $\alpha_c = \text{apply}((\mathcal{C}, S), \top)$ , we can define the following *ready* operator to summarize eagerly the critical statements:

$$\text{ready}(\mathcal{C}, S) := (\mathcal{C} \sqcup \bigsqcup^c \{ \text{trans}(\alpha_c, \text{stmt}) \mid \text{stmt} \in S_{|\mathcal{P}_{\text{trans}}(\alpha_c)} \}, S_{|\neg \mathcal{P}_{\text{trans}}(\alpha_c)})$$

where  $S_{|\mathcal{P}_{\text{trans}}(\alpha_c)} = \{ \text{stmt} \in S \mid \mathcal{P}_{\text{trans}}(\alpha_c, \text{stmt}) = \text{true} \}$  represents the set of critical statements that are not critical any more under the current context. The remaining statements are represented by  $S_{|\neg \mathcal{P}_{\text{trans}}(\alpha_c)}$ .

With *ready*, an invocation statement can be summarized as:

$$\text{sum}^h(\text{call proc}) := \text{ready}(\text{sum}^h(\Lambda(\text{proc})))$$

This will allow us to summarize the critical statements wherever their accumulated context is adequate.

**THEOREM 3.3.** *Hybrid Inlining with the *ready* operator is sound and as precise as a version of Hybrid Inlining that propagates all critical statements to the root procedures.*

The definition of  $\mathcal{P}_{\text{trans}}$  is important in Hybrid Inlining, since it is used to find critical statements and decide when to stop their propagation, but it does not reveal any operator to be used directly. For a specific analysis, however, it is often straightforward to construct the corresponding operators according to this definition. For instance, recall the example discussed in Section 2. Following the definition of  $\mathcal{P}_{\text{trans}}$ , we can easily identify that its virtual invocation statements are critical. For an adequate context, it is also simple to find out that the context is adequate when the receiver  $tx$  and all its aliased variables are not accessible outside the scope of  $bar1()$ .

Adequate contexts can help improve the performance of Hybrid Inlining since it avoids unnecessary propagation of statements. To give more flexibility on the trade-off between performance and precision, we introduce context-ready predicates.

**3.2.2 Context-Ready Predicates.** A context-ready predicate decides whether the current context is *enough* for a critical statement.

We define a *context-ready predicate*  $\Phi(\text{stmt}, \alpha)$ , which takes as input a statement and the current context and produces as output

*true* if the current context is *enough* (not necessarily *adequate*) for the underlying analysis. If  $\Phi$  evaluates to *true*, our analysis just summarizes the critical statements. Thus, we have:

$$\text{ready}(\mathcal{C}, S) := (\mathcal{C} \sqcup \bigsqcup^c \{ \text{trans}(\alpha_c, \text{stmt}) \mid \text{stmt} \in S_{|\Phi(\alpha_c)} \}, S_{|\neg \Phi(\alpha_c)})$$

Compared to adequate contexts, such context-ready predicates may bring precision loss. However, it gains performance by stop propagating critical statements earlier. Still, it is more precise than conventional compositional analysis which abstracts all critical statements with  $\top$ . Context-ready predicates provide a way to fine-tuning the trade-off between performance and precision. Under this definition,  $\mathcal{P}_{\text{trans}}(\alpha_c, \text{stmt})$  becomes a special context-ready predicate, called an *adequate context predicate* and denoted  $\Phi_a$ .

Context-ready predicates are vital for the performance of Hybrid Inlining. Many industrial applications can have call stacks with over 100 callsites, and propagating all critical statements to the root procedure would cause unnecessary performance loss. By utilizing context-ready predicates, Hybrid Inlining enables a compositional analysis without requiring knowledge of the call string from root procedures during summarization. This allows for easy parallelization and incrementalization of our analysis.

We tend to always choose adequate context predicates as the context-ready predicates. However, we can also restrict the maximal propagation steps for each critical statement as  $k$  (similar to  $k$ -callsite-sensitivity in top-down analysis). This results in *k-limited* Hybrid Inlining, in practice.

**3.2.3 Analysis Process and Recursion Handling.** Our analysis is simple, as it just summarizes all reachable procedures with  $\text{sum}^h$  and then evaluates all the root procedures with *apply*. However, as revealed by the algorithm of  $\text{sum}^h$ , the summarization of a procedure needs to use the hybrid summaries in its called procedures. Since all procedures are analyzed in a random order (unlike in other works, our analysis does not build a call graph via a pre-analysis), it can happen that some callees are not summarized yet when a caller is to be summarized. In this case, our analysis just turns to summarize all the callees first, by adopting a simple depth-first search algorithm. However, we need to specifically handle call string recursion to ensure the termination of our algorithm. During the depth-first search, our algorithm maintains the accumulated call string. Once recursion is detected on the call string, our analysis will compute its summary with a least fix point operator after unrolling it twice.

**3.2.4 Workflow of Applying Hybrid Inlining.** Our Hybrid Inlining framework is general and can be applied to any compositional analysis, regardless of the programming languages used. To apply Hybrid Inlining, users only need to find critical statements and define their context-ready predicates.

We have given the definition of  $\mathcal{P}_{\text{trans}}$ , following which, users can identify what kinds of statements are critical for a given analysis. For instance, in Section 2, all virtual invocation statements are identified as critical. In general, critical statements can thus be found via simple syntactic pattern matching.

For context-ready predicates, users can easily define trivial predicates like  $k$ -limited ones. However, if users want to adopt adequate context predicates, then they should construct an operator that determines whether a context is adequate, following the definition

of  $\mathcal{P}_{\text{trans}}$ . In the example from Section 2, it is also trivial to find that the context is adequate when the receiver of a virtual invocation and all its aliased variables are not accessible outside. This can therefore be easily implemented as an operator.

From the implementation perspective, users will only need to perform three tasks: (1) embedding the summary with statements, (2) adding a mechanism for summarizing statements lazily, (3) adding a function determining whether a statement should be summarized. These changes should be within hundreds of lines of code.

## 4 INSTANTIATION

In this section, we instantiate our framework on several analyses, including a pointer analysis, which has been illustrated in Section 2.

### 4.1 Instantiation on A Pointer Analysis

Pointer analysis is fundamental for many client analyses. However, compared to its top-down counterpart, compositional pointer analysis is much less explored in the literature. Thus, we design a compositional and flow-insensitive pointer analysis first and then show how to apply Hybrid Inlining to it.

The pointer analysis follows conventional Andersen-style, but with a special constraint solving process making it compositional.

**4.1.1 Constraint Generation.** We describe our pointer analysis with respect to the following language:

```

proc  := stmt+
stmt  := lv = expr | call proc(lv0-n)
lv    := v | lv.f | lv[c] | lv[v]      f ∈ F   v ∈ V
expr  := lv | c | l                  c ∈ C   l ∈ L

```

A procedure  $\text{proc}$  is defined as a sequence of assignment and procedure invocation statements. A left value  $\text{lv}$  can be a variable  $v \in \mathbb{V}$ , a field access  $\text{lv}.f$  (where  $\mathbb{F} \ni f$  denotes the set of fields) or an index access (i.e.,  $\text{lv}[c]$  or  $\text{lv}[v]$ ). An index can be a constant  $c \in \mathbb{C}$  (e.g., string or integer) or a variable, allowing us to model arrays and maps. A right-value expression can be a left value, a constant or a location (i.e., an allocation site  $l \in \mathbb{L}$ ). We use  $\omega \in \mathbb{V} \times (\mathbb{F} \cup \mathbb{C})^*$  to denote an access path.

**Abstract states and summaries.** An abstract state  $\mathbf{a} \in \mathfrak{A}$  is a set of set constraints on access paths of the following form:

$$\omega \supseteq \omega \quad \omega \supseteq \{l\} \quad \omega \supseteq \{c\}$$

This definition of  $\mathbf{a}$  is standard for Anderson-style pointer analysis. An abstract summary  $\mathbf{c} : \mathfrak{A} \Rightarrow \mathfrak{A}$  is defined as  $\mathbf{a} \mapsto \mathbf{a} \cup \mathbf{a}_n$ . That is, for each precondition  $\mathbf{a}$ , a summary just adds new set constraints in  $\mathbf{a}_n$  (which are generated from statements).

**Abstract transfer functions.** The abstract transfer function  $\text{trans}$  transforms a statement into a summary. As shown in Figure 4, the transfer function is defined in terms of a constraint generation function  $\text{cons} : \mathfrak{A} \times \mathbb{S} \Rightarrow \mathfrak{A}$ , which transforms a statement into a set of constraints with a given context.

For an assignment statement,  $\text{cons}$  just maps the left/right values to access paths (via  $\text{eval}$ ) and generates set relations (i.e.,  $\supseteq$ ) on them. The definition of  $\text{eval}$  is straightforward, except for the case of  $\text{eval}(\text{lv}[v])(\mathbf{a})$ , where a variable is used as an index. In this case, if the points-to set  $\text{pt}(v, \mathbf{a})$  of  $v$  only contains constants, then each constant in  $\text{pt}(v, \mathbf{a})$  is taken as an index. Otherwise, a special symbol  $\pi$  is introduced to represent an undecidable index. Note

**Evaluation**  $\text{eval} : \text{lv} \Rightarrow \mathcal{P}(\mathbb{V} \times (\mathbb{F} \cup \mathbb{C})^*)$

- (1)  $\text{eval}(v)(\mathbf{a}) := \{v\}$
- (2)  $\text{eval}(\text{lv}.f)(\mathbf{a}) := \{\omega.f \mid \omega \in \text{eval}(\text{lv})\}$
- (3)  $\text{eval}(\text{lv}[c])(\mathbf{a}) := \{\omega.c \mid \omega \in \text{eval}(\text{lv})\}$
- (4)  $\text{eval}(\text{lv}[v])(\mathbf{a}) := \{\omega.c \mid \omega \in \text{eval}(\text{lv}), c \in \text{pt}(v, \mathbf{a})\}$   
if  $\text{pt}(v, \mathbf{a}) \subseteq \mathbb{C}$
- (5)  $\text{eval}(\text{lv}[v])(\mathbf{a}) := \{\omega.\pi \mid \omega \in \text{eval}(\text{lv})\}$   
if  $\text{pt}(v, \mathbf{a}) \not\subseteq \mathbb{C}$

**Constraint Generation**  $\text{cons} : (\mathfrak{A} \times \mathbb{S}) \Rightarrow \mathfrak{A}$

- (6)  $\text{cons}(\mathbf{a}, \text{lv}_0 = \text{lv}_1) := \{\omega_0 \supseteq \omega_1 \mid \omega_i \in \text{eval}(\text{lv}_i, \mathbf{a}), i \in \{0, 1\}\}$
- (7)  $\text{cons}(\mathbf{a}, \text{lv} = c) := \{\omega \supseteq \{c\} \mid \omega \in \text{eval}(\text{lv}, \mathbf{a})\}$
- (8)  $\text{cons}(\mathbf{a}, \text{lv} = l) := \{\omega \supseteq \{l\} \mid \omega \in \text{eval}(\text{lv}, \mathbf{a})\}$
- (9)  $\text{cons}(\mathbf{a}, S) := \bigcup_{\text{stmt} \in S} \text{cons}(\mathbf{a}, \text{stmt})$
- (10)  $\text{cons}(\mathbf{a}, \text{call proc}(\text{lv}_{0-n})) := \bigcup_{\text{proc}' \in \text{dispatch}(\mathbf{a}, \text{proc}(\text{lv}_{0-n}))} \text{cons}(\mathbf{a}, \Lambda(\text{proc}'(\text{lv}_{0-n})))$

**Transfer Functions**  $\text{trans} : (\mathfrak{A} \times \mathbb{S}) \Rightarrow (\mathfrak{A} \Rightarrow \mathfrak{A})$

- (11)  $\text{trans}(\mathbf{a}, \text{stmt}) := \mathbf{a}' \mapsto \mathbf{a} \sqcup \text{cons}(\mathbf{a}, \text{stmt})$

Figure 4: Abstract semantics of a pointer analysis.

```

1  getP(map, key){      4  setP(map, key, v){      7  build(u){
2  return map[key];    5  map[key]=v;          8  v = new;
3  }                   6  }                  9  setP(v, "old",
                                10  getP(u, "cur"));
                                11 return v;}

```

Figure 5: Illustrating Hybrid Inlining on the pointer analysis.

that the points-to set  $\text{pt}(v, \mathbf{a})$  in our analysis may contain access paths involving non-local variables, which will be illustrated in Section 4.1.2. The definition of  $\text{cons}$  is also lifted to deal with sets of statements  $S$ , as in Definition (9) of Figure 4.

For a procedure invocation statement  $\text{call proc}(\text{lv}_{0-n})$ ,  $\text{cons}$  first finds all implementations of  $\text{proc}$  via dispatching (denoted as  $\text{dispatch}(\mathbf{a}, \text{proc})$ ). Then for each implementation  $\text{proc}'$ ,  $\text{cons}$  collects the constraints from all the statements  $\Lambda(\text{proc}')$ .

The join operator  $\sqcup$  on summaries is defined as:

$$(\mathbf{a} \mapsto \mathbf{a} \sqcup \mathbf{a}_0) \sqcup (\mathbf{a} \mapsto \mathbf{a} \sqcup \mathbf{a}_1) := \mathbf{a} \mapsto \mathbf{a} \sqcup (\mathbf{a}_0 \sqcup \mathbf{a}_1)$$

We omit the context in the definition of this join operator, because the context makes no difference on its semantics.

The abstract semantics of this compositional pointer analysis can be derived from the aforementioned operators by following the principles defined in Section 3.1.

**4.1.2 Constraint Solving.** This compositional pointer analysis is a kind of partial program analysis. When a procedure is summarized, the points-to sets of some variables (e.g., the parameters) in the procedure can be unknown. Thus, the constraint solving process in our analysis is different from that in conventional top-down analysis in the sense that not only allocation sites but also non-local variables (e.g., the parameters) are propagated along the constraints. In this case, the points-to set  $\text{pt}(v, \mathbf{a})$  of a variable  $v$  can include allocation sites and non-local variables, as illustrated in Section 2.1.

Even though this pointer analysis is context-sensitive to some extent, it loses context-sensitivity on two kinds of statements: assignments that involve  $\text{lv}[v]$  and virtual procedure invocations. For instance,  $\text{cons}(\mathbf{a}, \text{call proc}(\text{lv}_{0-n}))$  in Definition (10) will collect

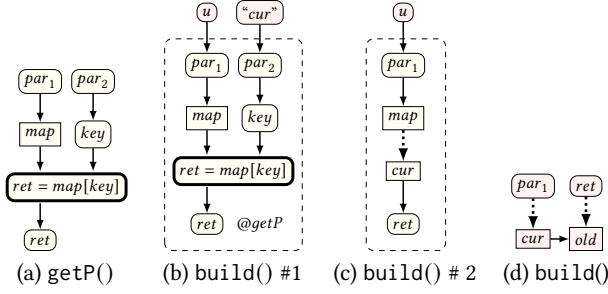


Figure 6: Constraints generated for the code in Figure 5.

the constraints from all possible implementations of the invoked virtual procedure when  $\alpha = \top$ . This incurs a precision loss. An alternative is to generate disjunction here in order to be strictly context-sensitive. However, this brings a significant performance loss. Furthermore, it is just impossible to generate disjunction on some critical statements, like those involving  $lv[v]$ , since the points-to set of the variable  $v$  is unbounded when the context is  $\top$ .

Hybrid Inlining can make it strictly context-sensitive on these two kinds of critical statements.

**4.1.3 Hybrid Inlining for Pointer Analysis.** To apply Hybrid Inlining in our framework (Section 3), we just need to identify the critical statements following  $\mathcal{P}_{trans}$  and define context-ready predicates.

**Critical statements.** In this pointer analysis, it is straightforward to find out whether a statement is critical from the definitions of their transfer functions: a statement is *critical/non-critical* if the context *does/does-not* make a difference on the generated constraints. Following this principle, Definitions (4), (5) and (10) in Figure 4 indicate that assignment statements that involve  $lv[v]$  and virtual procedure invocation statements (i.e.,  $call\ proc$  where  $|dispatch(\top, proc)| > 1$ ) are critical. All the others are non-critical. The two kinds of critical statements can be easily identified with syntactic pattern matching and class hierarchy analysis.

**Context-ready predicates.** It is straightforward to define a context-ready predicate  $\Phi$  for  $k$ -limited Hybrid Inlining (by propagating every critical statement in  $k$  steps). However, we can also define adequate context predicates  $\Phi_a$  (by propagating every critical statement the number of steps as needed) according to  $\mathcal{P}_{trans}$ . An adequate context-ready predicate for statements involving index access (i.e.  $lv[v]$ ) can be defined as follows:

$$\Phi_a(lv[v] = expr, \alpha) := pt(v, \alpha) \cap free(\alpha) = \emptyset$$

where  $free(\alpha)$  denotes the set of variables that are accessible outside the current procedure. This predicate indicates that the propagation stops when the points-to set of the index variable  $v$  does not contain any free variables, i.e., it cannot change any more. Thus, this context is adequate. Note that the variables in the critical statements in the hybrid summary are also free variables.

We also need an adequate context predicate  $\Phi_a$  for virtual invocation statements (i.e.  $lv = proc(lv_0-n)$ ), which is defined as:

$$\Phi_a(lv = proc(lv_0-n), \alpha) := pt(lv_0, \alpha) \cap free(\alpha) = \emptyset \\ \vee |dispatch(proc, \alpha)| = 1$$

This predicate checks whether the points-to set of the receiver  $lv_0$  contains free variables. It also checks whether the callsite is

monomorphic, which can happen when the type information gathered on the receiver has indicated such a unique implementation. This condition is adequate if the analysis does not consider error states (when, e.g., the receiver points to *null*).

These two adequate context predicates can both be implemented as operators.

**Example 4.1.** In Section 2.2, we have discussed the effect of Hybrid Inlining on virtual invocation statements. In this example, we consider the case on statements involving index access. Figure 5 gives a code segment, where two *map* accesses at lines 2 and 5 are wrapped by the two procedures *getP()* and *setP()*. Without Hybrid Inlining, our pointer analysis would just totally lose context- and index-sensitivity on these *map* accesses, because the value of the index (stored in the variable *key*) is unknown when *getP()* and *setP()* are summarized. Note that it is impossible to create disjunction here, because the possible values stored in *key* are unbounded.

With Hybrid Inlining, our analysis identifies  $ret@getP=map[key]$  as critical (by syntactic checking), and decides to propagate it, because the points-to set of *key* includes the free variable  $par_2@getP$  as shown in Figure 6(a). By inlining this summary to the procedure *build()*, our analysis produces what is shown in Figure 6(b) (only some constraints are shown for brevity). After cleaning  $par_2@getP$ , the points-to set of *key* only contains the constant “*cur*”. Thus, the critical statement is summarized, producing Figure 6(c). By applying a similar process to *setP()*, we obtain the summary of *build()* as shown in Figure 6(d). This summary achieves index- and context-sensitivity due to Hybrid Inlining.

## 4.2 Other Instantiations

We also instantiate our framework on string analysis, reflection analysis and taint analysis. The main principles are similar to that of the pointer analysis. Therefore, we only show how to find critical statements and define their adequate context predicates.

For string analysis, its critical statements are some string manipulation APIs (e.g., *append()* and *replace()*) and their adequate context predicates require all operands to be evaluated to constants.

For reflection analysis, its critical statements are some reflection APIs (e.g., *getClass()*) and the corresponding adequate context predicates are defined similarly to those for regular virtual call statements (handled in the pointer analysis instantiation).

For taint analysis, it works on top of the underlying pointer analysis so that it shares the same critical statements and adequate context predicates as the pointer analysis.

## 5 IMPLEMENTATION AND EVALUATION

We have implemented our compositional pointer/string/reflection/-taint analyses in Java based on GraalVM [35], which transforms Java bytecode to an SSA-like graph representation. That infuses some degree of flow-insensitivity into our analyses. We have implemented Hybrid Inlining with both  $k$ -limited propagation steps and adequate context predicates.

Our evaluation focuses on pointer analysis, which is a fundamental analysis underpinning almost all other forms of analysis. We evaluate our approach by addressing three research questions:



**RQ1:** What are the benefits of applying Hybrid Inlining on conventional compositional pointer analysis with respect to precision and performance?

**RQ2:** Why does Hybrid Inlining achieve such benefits?

**RQ3:** What is the practical significance of our work when Hybrid Inlining is applied to industrial applications?

## 5.1 RQ1: Precision and Performance

We first examine the precision of Hybrid Inlining on a micro-benchmark suite, *PointerBench* [47], which comes with the ground truth on aliased variable pairs. Each test case contains several queries on whether two variables are aliased. All its test cases, including also the example in Figure 1, are listed in Table 1.

We also run the artifact from [19] on this benchmark suite, which provides both context-insensitive (“TopCI”) and configurable context-sensitive top-down pointer analyses. We have selected 3-callsite-sensitivity (i.e., “3C”), because that is the maximal context sensitivity needed on these micro-benchmarks. We also run our compositional pointer analysis (Column “ComCI”) and the same analysis extended to support 3-limited Hybrid Inlining (i.e., “HI3”), where each critical statement is propagated for 3 steps, and Hybrid Inlining with adequate context predicates (i.e., “HIA”).

In Table 1, each ✓ (resp. ⊗) represents a true (resp. false) positive. We denote the number of true/false positives by the same number of ✓’s/⊗’s. From the table, we can see that HI3 and HIA are the most precise while the top-down context-insensitive analysis (i.e., TopCI) is the most imprecise. However, precision cannot be just simply understood as a single value. This table also reveals where each analysis gains/loses precision. In fact, the most imprecise TopCI is more precise than HI3/HIA on some cases (e.g., *exception*), because it models control flows involving exceptions more precisely. HI3 is only more precise than 3C on the test cases in the *Containers* category, since our implementation models containers more precisely (as shown in Section 4). Context-sensitivity is actually only needed for the test cases, *returnValue*, *contextSensitivity*, *objectSensitivity* and *overviewExample*. For these test cases, 3C, HI3 and HIA lose no precision. In the case of *null*, no analysis reports true/false positives. Finally, neither of these analyses reports false negatives, which means that these analyses are all sound on this benchmark suite.

HI3 and HIA produce the same result and are thus listed in the same column. The adequate context predicate in HIA is as defined in Section 4.1.3. This definition ensures no precision loss, which is also demonstrated in this experiment. While HI3 and HIA share the same precision, HIA actually exhibits more performance potential, since it propagates critical statements in fewer steps in some cases. For instance, in the case of “*contextSensitivity*”, its critical statements are propagated by three steps fewer in HIA than in HI3. However, this performance gain is not obvious on this micro benchmark. We rely on large applications to evaluate the precision and performance difference of HIA and *k*-limited Hybrid Inlining.

To evaluate the performance and precision of Hybrid Inlining on large applications, we run all DaCapo [4] benchmarks in the artifact of [19] except *python*, because our analysis (even in a context-insensitive setting) just explodes with the number of access paths introduced. All experimental results are shown in Table 2. Our compositional pointer analysis is run with three different configurations:

**Table 1: Precision on PointerBench.**

| Progs        |                    | TopCI    | 3C       | ComCI    | HI3/HIA  |
|--------------|--------------------|----------|----------|----------|----------|
| Basic        | branching          | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | interprocedural    | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | loops              | ✓✓✓⊗     | ✓✓✓⊗     | ✓✓✓⊗     | ✓✓✓⊗     |
|              | parameter          | ✓✓✓✓     | ✓✓✓✓     | ✓✓✓✓     | ✓✓✓✓     |
|              | recursion          | ✓        | ✓        | ✓        | ✓        |
|              | returnValue        | 4 ✓, ⊗   | 4 ✓      | 4 ✓      | 4 ✓      |
| Containers   | simpleAlias        | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | array              | ✓✓⊗      | ✓✓⊗      | ✓✓       | ✓✓       |
|              | list               | 4 ✓, 2 ⊗ | 4 ✓, 2 ⊗ | 4 ✓, 2 ⊗ | 4 ✓, 2 ⊗ |
|              | map                | ✓✓⊗      | ✓✓⊗      | ✓✓       | ✓✓       |
| General Java | set                | ✓⊗       | ✓⊗       | ✓⊗       | ✓⊗       |
|              | accessPath         | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | fieldSensitivity   | ✓✓✓✓     | ✓✓✓✓     | ✓✓✓✓     | ✓✓✓✓     |
|              | contextSensitivity | 6 ✓, 3 ⊗ | 6 ✓,     | 6 ✓, 3 ⊗ | 6 ✓      |
|              | flowSensitivity    | ✓        | ✓        | ✓        | ✓        |
|              | objectSensitivity  | 4 ✓, 4 ⊗ | 4 ✓      | 4 ✓      | 4 ✓      |
| Corner Cases | strongUpdate       | ✓✓✓⊗     | ✓✓✓⊗     | ✓✓✓⊗     | ✓✓✓⊗     |
|              | exception          | ✓✓       | ✓✓       | ✓✓⊗      | ✓✓⊗      |
|              | interface          | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | null               |          |          |          |          |
|              | outerClass         | ✓✓⊗      | ✓✓⊗      | ✓✓⊗      | ✓✓⊗      |
|              | staticVariable     | ✓✓       | ✓✓       | ✓✓       | ✓✓       |
|              | superClass         | ✓✓⊗      | ✓✓⊗      | ✓✓⊗      | ✓✓⊗      |
|              | overviewExample    | ✓⊗       | ✓✓       | ✓⊗       | ✓✓       |
| Precision    |                    | 79.6%    | 86.8%    | 83.1%    | 88.1%    |

(1) context-insensitive (i.e., “ComCI”); (2) Hybrid Inlining where each critical statement is propagated in two steps (i.e., “HI2”); (3) Hybrid Inlining with adequate context predicates (i.e., “HIA”). In the third configuration, we have also implemented a simple heuristic to limit the propagation of some critical statements even if adequate context predicates are not satisfied for the sake of performance. If the number of critical statements reaches a threshold in a summary, our analysis summarizes the critical statements that may still need to propagate more steps immediately. This is to avoid the number of critical statements to explode in some extreme cases. All experiments are run on a personal laptop with a 2.6 GHz CPU and 16 GB memory. The time budget is set as one hour.

On each test case, we report the number of reachable procedures (i.e., “#reach”), the number of polymorphic call-sites (i.e., “#poly”) and running time (i.e., “time(s)”). Polymorphic call-sites are used for measuring precision (smaller is better) and running time is for performance. Table 2 shows that both HI2 and HIA are significantly more precise than ComCI with little performance loss. HI2 and HIA reduce the number of polymorphic call-sites by 10.4% and 17.6%, on average, respectively. The recorded running times are all within 2X of ComCI except for *xalan*. For some test cases such as *fop*, HI2 and HIA are even faster than ComCI. On average, HI2 and HIA introduce 27.0% and 64.5% additional overheads on running time, respectively. Compared with HIA and HI2, HIA requires more running time, but is significantly more precise than HI2. The number of polymorphic call-sites is known [26] to be hard to reduce. The precision advantage of HIA over HI2 shows both the effectiveness of HIA and the importance of context-sensitivity on precision.

**Table 2: Performance and precision on DaCapo.**

| Metric  | T.      | ComCI | HI2   | HIA   | T.       | ComCI | HI2   | HIA   |
|---------|---------|-------|-------|-------|----------|-------|-------|-------|
| #reach  | antlr   | 5280  | 5173  | 5129  | bloat    | 8233  | 7657  | 7554  |
| #poly   |         | 1891  | 1786  | 1692  |          | 2627  | 2285  | 2066  |
| time(s) |         | 11.6  | 13.5  | 13.4  |          | 18.9  | 22.5  | 27.4  |
| #reach  | chart   | 16187 | 15835 | 15827 | eclipse  | 23033 | 20829 | 19218 |
| #poly   |         | 6529  | 5939  | 5503  |          | 13535 | 12583 | 9602  |
| time(s) |         | 41.2  | 48.3  | 62.8  |          | 102.1 | 129.0 | 200.0 |
| #reach  | fop     | 19302 | 16937 | 14461 | hsqldb   | 7785  | 7708  | 7695  |
| #poly   |         | 7530  | 5295  | 4187  |          | 1902  | 1798  | 1718  |
| time(s) |         | 66.4  | 60.1  | 49.4  |          | 16.0  | 22.8  | 30.5  |
| #reach  | luindex | 7548  | 7426  | 7391  | lusearch | 6767  | 6659  | 6646  |
| #poly   |         | 2682  | 2505  | 2400  |          | 2245  | 2061  | 2011  |
| time(s) |         | 13.3  | 16.4  | 15.1  |          | 19.1  | 18.2  | 27.8  |
| #reach  | pmd     | 10396 | 9945  | 9879  | xalan    | 14240 | 12814 | 12190 |
| #poly   |         | 2806  | 2546  | 2459  |          | 6781  | 6081  | 5965  |
| time(s) |         | 22.6  | 42.5  | 28.1  |          | 56.4  | 85.1  | 218.9 |

**Table 3: Critical statements and their propagation**

| Prog     | #C    | #C <sub>prop</sub> | %C <sub>prop</sub> | K <sub>max</sub> | K <sub>avg</sub> |
|----------|-------|--------------------|--------------------|------------------|------------------|
| antlr    | 3854  | 2224               | 0.93%              | 10               | 3.25             |
| bloat    | 6443  | 3353               | 1.20%              | 17               | 4.57             |
| chart    | 19512 | 15430              | 2.57%              | 9                | 2.52             |
| eclipse  | 28350 | 12341              | 1.43%              | 26               | 3.30             |
| fop      | 16433 | 9293               | 1.77%              | 10               | 1.71             |
| hsqldb   | 6651  | 2574               | 0.87%              | 6                | 1.82             |
| luindex  | 6668  | 3747               | 1.44%              | 19               | 2.81             |
| lusearch | 5909  | 4166               | 1.82%              | 13               | 2.55             |
| pmd      | 6909  | 3102               | 0.90%              | 10               | 2.11             |
| xalan    | 57055 | 10594              | 1.89%              | 38               | 5.22             |

Hybrid inlining can be seen an application of selective context-sensitivity on compositional analysis. To the best of our knowledge, our approach is the first on selective context sensitivity for compositional analysis. However, it is not directly comparable to the selective context-sensitive top-down pointer analyses [26, 28]. Our approach and these existing ones are built on different grounds, i.e., compositional analysis and top-down analysis, respectively. Some features (e.g., field sensitivity and library handling) of the underlying compositional/top-down pointer analysis implementations can affect precision more than selective context sensitivity techniques. It is known to be difficult to separate their respective effects on precision. Furthermore, top-down analysis is often *unsound* on the points-to set of the receiver of an invocation in handling program features such as reflection and native methods. This unsoundness is observed [19, 36] on the current popular top-down pointer analysis platforms [22, 50]. Compositional analysis can achieve soundness by assuming all possibilities when it is not sure about the types of receivers. This explains why we do not include TopCI or 2C in our experiments, since even TopCI and ComCI are not comparable.

## 5.2 RQ2: Result Analysis

To understand how Hybrid Inlining achieves such benefits, we provide some insights on how critical statements are propagated. Table 3 shows the number of critical statements (Column “#C”),

the number of critical statements propagated (“#C<sub>prop</sub>”), the ratio of such critical statements over all statements (Column “%C<sub>prop</sub>” counted in the Graal IR), and the maximal and average number of propagation steps (Columns “K<sub>max</sub>” and “K<sub>avg</sub>”) in the HIA configuration (i.e., Hybrid Inlining with adequate context predicates).

We can see that the critical statements that need to be propagated only account for a very small portion of all statements (less than 2% in all test cases except *chart*). This explains why Hybrid Inlining delivers excellent performance: it only re-analyzes critical statements rather than non-critical statements for different contexts. As for the number of propagation steps taken, while Hybrid Inlining allows individual critical statements to be propagated really far (up to 38 steps), the average number of propagation steps is between 1.71 and 5.22. This explains why HIA does not cause too much performance loss, and why HIA is more precise than HI2.

The ratio of critical statements plays an important role in the performance of Hybrid Inlining. In the worst case, if the ratio is 100%, then Hybrid Inlining boils down to full inlining. For all our implemented analyses, the ratio is very low. Taint analysis shares the same ratio of critical statements with pointer analysis (since it is built on pointer analysis). Reflection and string analyses have even lower ratios since only some special APIs are critical.

## 5.3 RQ3: Applications in the Industry

We evaluate our work on 10 real industrial applications from Ant Group, which are the core modules of a large financial transaction system. These applications are all related to the microservice architecture, with each consisting of dozens or hundreds of entry procedures. Due to the challenges in creating a reliable single main procedure for top-down analysis, we exclusively employ compositional pointer analysis by traversing all application procedures.

Table 4 shows the results for different applications (anonymized in the “Prog” column). These applications heavily utilize libraries, which are typically over ten times larger than the application code. To ensure termination, we restrict the maximum call string depth to 3 when dealing with library procedures. We consider two configurations: context-insensitive (i.e., “ComCI”) and Hybrid Inlining (i.e., “HIA”). We can see that while gaining precision, HIA is surprisingly faster than ComCI in 6 out of these 10 applications. The additional time overhead brought by HIA is only 1% on average. There are two reasons for this: (1) the portion of propagated critical statements is very low. This is because these industrial applications deal with much simpler tasks (like transactions) than the aforementioned open-source applications; and (2) these industrial applications use heavily callbacks (e.g., with TransactionTemplate in the *Spring* framework). Imprecise dispatching on their virtual call-sites can lead to huge performance loss (and precision loss).

Our taint analysis (running on top of our pointer analysis) has been deployed as a microservice in Ant Group to achieve different business objectives, including: (1) the quality assurance team is interested in what field in the input can affect a given branch condition, which facilitates automatic testing; (2) the account reconciliation team is interested in how the values involving money may flow across the microservices; and (3) the SRE (Site Reliability Engineer) team is interested in how dynamic configurations influence the behaviors of certain applications online.

**Table 4: Performance on industrial applications.**

| Prog  | ComCI  |       |         | HIA    |       |         |                    |
|-------|--------|-------|---------|--------|-------|---------|--------------------|
|       | #reach | #poly | time(s) | #reach | #poly | time(s) | %C <sub>prop</sub> |
| app1  | 7428   | 178   | 9.6     | 7414   | 127   | 9.2     | 0.29               |
| app2  | 82162  | 1304  | 91.2    | 80728  | 1233  | 101.5   | 0.24               |
| app3  | 23994  | 559   | 26.9    | 23989  | 532   | 22.8    | 0.15               |
| app4  | 25451  | 937   | 28.3    | 25436  | 839   | 34.8    | 0.21               |
| app5  | 24903  | 1060  | 24.3    | 24901  | 526   | 27.9    | 0.47               |
| app6  | 38805  | 468   | 51.9    | 38773  | 399   | 51.2    | 0.10               |
| app7  | 30724  | 295   | 37.9    | 30705  | 247   | 40.1    | 0.19               |
| app8  | 43627  | 3355  | 94.0    | 43612  | 2824  | 85.1    | 0.38               |
| app9  | 15252  | 277   | 22.6    | 15289  | 242   | 22.4    | 0.16               |
| app10 | 11880  | 138   | 12.7    | 11876  | 111   | 11.1    | 0.20               |

Our implementation has become the basis for several platforms (e.g., the automatic testing platform) inside Ant Group, and is called thousands of times each day. This demonstrates the effectiveness of our approach. From our experience, context-sensitivity is most needed in analyzing callbacks and index accesses on Map/Json. Callbacks are either implemented by users or with libraries like *TransactionTemplate* in the *Spring* framework. Index accesses on Map/Json are often with constant keys (which are surprisingly ubiquitous because many developers tend to utilize maps to hold added data items as the systems evolve). Our framework actually fits into this scenario well, where these critical statements only account for a small amount of code but can be wrapped deeply (more than 3 layers). We constantly receive feed-backs (e.g., bug fixes and new library modelling requests) from the users, which help us improve our design and implementation over time.

#### 5.4 Threat to Validity

The main threat to the validity of our evaluation is whether the ratio of critical statements over all statements in a program discovered in our four implemented analyses is as low as in other instantiated analyses. We have attempted to mitigate this threat to validity by evaluating four representative instantiations on both open-source benchmarks and real-world industrial applications.

One can easily get the impression that critical statements are not very rare, like the virtual invocation statements encountered during the pointer analysis. However, the ratio of critical statements is actually computed in an intermediate representation (operated on by our analyzers), which often exceeds that in the corresponding source code. In practice, the ratio of critical statements seems to be low (at least as detected in our evaluation).

Even in cases when the ratio of critical statements can be relatively high, we can still apply  $k$ -limited Hybrid Inlining or consider only the statements that affect precision the most as being critical, to make a trade-off between performance and precision.

## 6 RELATED WORK

Context-sensitivity on inter-procedural analysis has been an important research topic for many years. We briefly review some of the past research efforts on this. Inter-procedural analysis can be classified into top-down analysis and compositional analysis.

Inlining is a conventional way to perform inter-procedural top-down analysis [9]. However, full inlining can hardly scale up. Thus,

calling contexts are often limited. For instance, in  $k$ -CFA [16, 44], the length of a call string is limited to be  $k$ . Various representations for adopting limited contexts have been proposed, including  $k$ -object [33, 45],  $k$ -type [45], and  $k$ -this [31]. A popular way for implementing such an analysis is to use BDDs [3, 52] and Datalog [5]. Some work models inter-procedural analysis as a reachability problem [37, 39, 56], which can be solved with a pushdown system [38]. Non-trivial analyses exist, including pointer analysis [7, 46, 54, 55] and taint analysis [2] with field sensitivity.

Given the large sizes of current industrial software applications, top-down inter-procedural analysis can hardly scale up with high context-sensitivity (i.e. a relatively large  $k$  even under  $k$ -limiting). Many optimizations have been considered, including parallelization [1], demand-driven analysis [20, 41, 48], selective context-sensitivity [18, 23, 26–29], and incremental analysis [30, 40]. Another useful optimization [51, 58] utilizes disks to store the data-flow facts generated in order to reduce memory usage.

Compositional analysis has demonstrated scalability in industry [6, 14] by analyzing each procedure only once and enabling incrementalization [6] and parallelization [43]. Theoretical investigations have been conducted in [10]. Recent advancements primarily focus on designing new symbolic abstract domains for summarizing pointer information [8, 13, 34, 53] and shape properties [17, 21]. Symbolic numeric domains like Polyhedra [12] also enable compositional analysis. Compositional analyses typically collaborate with top-down analyses for tasks such as call graph construction, property checking, and abstract state refinement [21, 34, 42, 57]. These approaches are distinct from our work, as theirs (resp. ours) focus on *how* (resp. *when*) to summarize statements.

Some work [57] combines top-down and compositional analyses, which can be seen as an approach on disjunction management. Some other work [25] utilizes top-down information to guide the splitting on disjuncts. Our approach differs from these works in two aspects: (1) our analysis propagates critical statements rather than disjunction of summaries; and (2) critical statements are not always propagated to the root procedures, meaning that our analysis does not always need information from the top (as the case of [57]) to achieve precise summaries of critical statements. As demonstrated in our evaluation, most critical statements do not propagate far.

Our work can be seen as an approach on selective context-sensitivity. This topic has been well explored for top-down analysis [23, 26, 27, 29]. However, unlike all these existing works, our work represents the first attempt on compositional analysis. Another difference is that context-sensitivity in our approach is more fine-grained at the level of statements rather than procedures.

## 7 CONCLUSION

We present a Hybrid Inlining framework, which inlines both the critical statements and the summaries of non-critical statements at each callsite. Our work represents the first attempt to apply selective context-sensitivity to compositional analysis. In addition, our approach supports context sensitivity at the level of statements rather than procedures, which reduces analysis time. The pointer analysis instantiated from our framework supports context-sensitivity with little performance loss (by incurring only 65% additional time overhead on DaCapo benchmarks and 1% on industrial applications).



## REFERENCES

- [1] Aws Albarghouthi, Rahul Kumar, Aditya V Nori, and Sriram K Rajamani. 2012. Parallelizing top-down interprocedural analyses. *ACM SIGPLAN Notices* 47, 6 (2012), 217–228.
- [2] Steven Arzt, Siegfried Rasthofer, Christian Fritz, Eric Bodden, Alexandre Bartel, Jacques Klein, Yves Le Traon, Damien Oeteau, and Patrick McDaniel. 2014. Flowdroid: Precise context, flow, field, object-sensitive and lifecycle-aware taint analysis for android apps. *Acm Sigplan Notices* 49, 6 (2014), 259–269.
- [3] Marc Berndt, Ondrej Lhoták, Feng Qian, Laurie Hendren, and Navindra Umanee. 2003. Points-to analysis using BDDs. *ACM SIGPLAN Notices* 38, 5 (2003), 103–114.
- [4] Stephen M Blackburn, Robin Garner, Chris Hoffmann, Asjad M Khang, Kathryn S McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z Guyer, et al. 2006. The DaCapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st annual ACM SIGPLAN conference on Object-oriented programming systems, languages, and applications*. 169–190.
- [5] Martin Bravenboer and Yannis Smaragdakis. 2009. Strictly declarative specification of sophisticated points-to analyses. In *Proceedings of the 24th ACM SIGPLAN conference on Object oriented programming systems languages and applications*. 243–262.
- [6] Cristiano Calcagno, Dino Distefano, Peter O’Hearn, and Hongseok Yang. 2009. Compositional shape analysis by means of bi-abduction. In *Proceedings of the 36th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 289–300.
- [7] Krishnendu Chatterjee, Bhavya Choudhary, and Andreas Pavlogiannis. 2017. Optimal Dyck reachability for data-dependence and alias analysis. *Proceedings of the ACM on Programming Languages* 2, POPL (2017), 1–30.
- [8] Ramkrishna Chatterjee, Barbara G Ryder, and William A Landi. 1999. Relevant context inference. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 133–146.
- [9] Patrick Cousot and Radhia Cousot. 1977. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 238–252.
- [10] Patrick Cousot and Radhia Cousot. 2001. Compositional separate modular static analysis of programs by abstract interpretation. In *Proc. SSGRR*. 6–10.
- [11] Patrick Cousot and Radhia Cousot. 2002. Modular static program analysis. In *International Conference on Compiler Construction*. Springer, 159–179.
- [12] Patrick Cousot and Nicolas Halbwachs. 1978. Automatic discovery of linear restraints among variables of a program. In *Proceedings of the 5th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*. 84–96.
- [13] Isil Dillig, Thomas Dillig, Alex Aiken, and Mooly Sagiv. 2011. Precise and compact modular procedure summaries for heap manipulating programs. *ACM SIGPLAN Notices* 46, 6 (2011), 567–577.
- [14] Dino Distefano, Manuel Fähndrich, Francesco Logozzo, and Peter W O’Hearn. 2019. Scaling static analyses at Facebook. *Commun. ACM* 62, 8 (2019), 62–70.
- [15] Yu Feng, Xinyu Wang, Isil Dillig, and Thomas Dillig. 2015. Bottom-up context-sensitive pointer analysis for Java. In *Asian Symposium on Programming Languages and Systems*. Springer, 465–484.
- [16] David Grove, Greg DeFouw, Jeffrey Dean, and Craig Chambers. 1997. Call graph construction in object-oriented languages. In *Proceedings of the 12th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 108–124.
- [17] Bhargav S Gulavani, Supratik Chakraborty, Ganesan Ramalingam, and Aditya V Nori. 2011. Bottom-up shape analysis using lisf. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 33, 5 (2011), 1–41.
- [18] Dongjie He, Jingbo Lu, and Jingling Xue. 2021. Context Debloating for Object-Sensitive Pointer Analysis. In *2021 36th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, New York, NY, USA, 79–91.
- [19] Dongjie He, Jingbo Lu, and Jingling Xue. 2022. Qilin: A New Framework for Supporting Fine-Grained Context-Sensitivity in Java Pointer Analysis. In *36th European Conference on Object-Oriented Programming (ECOOP 2022)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany.
- [20] Nevin Heintze and Olivier Tardieu. 2001. Demand-driven pointer analysis. *ACM SIGPLAN Notices* 36, 5 (2001), 24–34.
- [21] Hugo Illous, Matthieu Lemerre, and Xavier Rival. 2020. Interprocedural Shape Analysis Using Separation Logic-Based Transformer Summaries. In *International Static Analysis Symposium*. Springer, 248–273.
- [22] S. J. Fink. T. J. 2023. *Watson libraries for analysis (WALA)*. <http://wala.sourceforge.net>
- [23] Sehun Jeong, Minseok Jeon, Sungdeok Cha, and Hakjoo Oh. 2017. Data-driven context-sensitivity for points-to analysis. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–28.
- [24] Chris Lattner, Andrew Lenharth, and Vikram Adve. 2007. Making Context-Sensitive Points-to Analysis with Heap Cloning Practical For The Real World. In *Proceedings of the 2007 ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI’07)*. San Diego, California.
- [25] Huisong Li, Francois Berenger, Bor-Yuh Evan Chang, and Xavier Rival. 2017. Semantic-directed clumping of disjunctive abstract states. *ACM SIGPLAN Notices* 52, 1 (2017), 32–45.
- [26] Yue Li, Tian Tan, Anders Möller, and Yannis Smaragdakis. 2018. Precision-guided context sensitivity for pointer analysis. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (2018), 1–29.
- [27] Yue Li, Tian Tan, Anders Möller, and Yannis Smaragdakis. 2020. A principled approach to selective context sensitivity for pointer analysis. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 42, 2 (2020), 1–40.
- [28] Jingbo Lu, Dongjie He, and Jingling Xue. 2021. Selective Context-Sensitivity for k-CFA with CFL-Reachability. In *International Static Analysis Symposium*. Springer, 261–285.
- [29] Jingbo Lu and Jingling Xue. 2019. Precision-preserving yet fast object-sensitive pointer analysis with partial context sensitivity. *Proceedings of the ACM on Programming Languages* 3, OOPSLA (2019), 1–29.
- [30] Yi Lu, Lei Shang, Xinwei Xie, and Jingling Xue. 2013. An Incremental Points-to Analysis with CFL-Reachability. In *Compiler Construction - 22nd International Conference, CC 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16–24, 2013. Proceedings (Lecture Notes in Computer Science, Vol. 7791)*. Ranjit Jhala and Koen De Bosschere (Eds.). Springer, 61–81. [https://doi.org/10.1007/978-3-642-37051-9\\_4](https://doi.org/10.1007/978-3-642-37051-9_4)
- [31] Jonas Lundberg, Tobias Gutzmann, Marcus Edvinsson, and Welf Löwe. 2009. Fast and precise points-to analysis. *Information and Software Technology* 51, 10 (2009), 1428–1439.
- [32] Ravi Mangal, Mayur Naik, and Hongseok Yang. 2014. A correspondence between two approaches to interprocedural analysis in the presence of join. In *European Symposium on Programming Languages and Systems*. Springer, 513–533.
- [33] Ana Milanova, Atanas Rountev, and Barbara G Ryder. 2005. Parameterized object sensitivity for points-to analysis for Java. *ACM Transactions on Software Engineering and Methodology (TOSEM)* 14, 1 (2005), 1–41.
- [34] Erik M Nystrom, Hong-Seok Kim, and W Hwu Wen-mei. 2004. Bottom-up and top-down context-sensitive summary-based pointer analysis. In *International Static Analysis Symposium*. Springer, 165–180.
- [35] Oracle. 2021. GraalVM. <https://www.graalvm.org/>
- [36] Jyoti Prakash, Abhishek Tiwari, and Christian Hammer. 2021. Effects of Program Representation on Pointer Analyses—An Empirical Study. In *International Conference on Fundamental Approaches to Software Engineering*. Springer, Cham, 240–261.
- [37] Thomas Reps, Susan Horwitz, and Mooly Sagiv. 1995. Precise interprocedural dataflow analysis via graph reachability. In *Proceedings of the 22nd ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 49–61.
- [38] Thomas Reps, Stefan Schwoon, Somesh Jha, and David Melski. 2005. Weighted pushdown systems and their application to interprocedural dataflow analysis. *Science of Computer Programming* 58, 1-2 (2005), 206–263.
- [39] Mooly Sagiv, Thomas Reps, and Susan Horwitz. 1996. Precise interprocedural dataflow analysis with applications to constant propagation. *Theoretical Computer Science* 167, 1-2 (1996), 131–170.
- [40] Lei Shang, Yi Lu, and Jingling Xue. 2012. Fast and precise points-to analysis with incremental CFL-reachability summarisation: preliminary experience. In *2012 Proceedings of the 27th IEEE/ACM International Conference on Automated Software Engineering*. IEEE, 270–273.
- [41] Lei Shang, Xinwei Xie, and Jingling Xue. 2012. On-demand dynamic summary-based points-to analysis. In *Proceedings of the Tenth International Symposium on Code Generation and Optimization*. Association for Computing Machinery, New York, NY, USA, 264–274.
- [42] Qingkai Shi, Xiao Xiao, Rongxin Wu, Jinguo Zhou, Gang Fan, and Charles Zhang. 2018. Pinpoint: Fast and precise sparse value flow analysis for million lines of code. In *Proceedings of the 39th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 693–706.
- [43] Qingkai Shi and Charles Zhang. 2020. Pipelining bottom-up data flow analysis. In *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering*. 835–847.
- [44] Olin Shivers. 1988. Control flow analysis in Scheme. In *Proceedings of the ACM SIGPLAN 1988 conference on Programming Language design and Implementation*. 164–174.
- [45] Yannis Smaragdakis, Martin Bravenboer, and Ondrej Lhoták. 2011. Pick your contexts well: understanding object-sensitivity. In *Proceedings of the 38th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. 17–30.
- [46] Johannes Späth, Karim Ali, and Eric Bodden. 2019. Context-, flow-, and field-sensitive data-flow analysis using synchronized pushdown systems. *Proceedings of the ACM on Programming Languages* 3, POPL (2019), 1–29.
- [47] Johannes Späth, Lisa Nguyen Quang Do, Karim Ali, and Eric Bodden. 2016. Boomerang: Demand-driven flow- and context-sensitive pointer analysis for java. In *30th European Conference on Object-Oriented Programming (ECOOP 2016)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [48] Manu Sridharan, Denis Gopan, Lexin Shan, and Rastislav Bodik. 2005. Demand-driven points-to analysis for Java. *ACM SIGPLAN Notices* 40, 10 (2005), 59–76.
- [49] Tian Tan, Yue Li, Xiaoxing Ma, Chang Xu, and Yannis Smaragdakis. 2021. Making pointer analysis more precise by unleashing the power of selective context



- sensitivity. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (2021), 1–27.
- [50] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. 2010. Soot: A Java bytecode optimization framework. In *CASCON First Decade High Impact Papers*. 214–224.
  - [51] Kai Wang, Aftab Hussain, Zhiqiang Zuo, Guoqing Xu, and Ardalan Amiri Sani. 2017. Graspan: A single-machine disk-based graph system for interprocedural static analyses of large-scale systems code. *ACM SIGARCH Computer Architecture News* 45, 1 (2017), 389–404.
  - [52] John Whaley and Monica S Lam. 2004. Cloning-based context-sensitive pointer alias analysis using binary decision diagrams. In *Proceedings of the ACM SIGPLAN 2004 conference on Programming Language Design and Implementation*. 131–144.
  - [53] John Whaley and Martin Rinard. 1999. Compositional pointer and escape analysis for Java programs. In *Proceedings of the 14th ACM SIGPLAN conference on Object-oriented programming, systems, languages, and applications*. 187–206.
  - [54] Guoqing Xu, Atanas Rountev, and Manu Sridharan. 2009. Scaling CFL-reachability-based points-to analysis using context-sensitive must-not-alias analysis. In *European Conference on Object-Oriented Programming*. Springer, 98–122.
  - [55] Qirun Zhang, Michael R Lyu, Hao Yuan, and Zhendong Su. 2013. Fast algorithms for Dyck-CFL-reachability with applications to alias analysis. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 435–446.
  - [56] Qirun Zhang and Zhendong Su. 2017. Context-sensitive data-dependence analysis via linear conjunctive language reachability. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. 344–358.
  - [57] Xin Zhang, Ravi Mangal, Mayur Naik, and Hongseok Yang. 2014. Hybrid top-down and bottom-up interprocedural analysis. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*. 249–258.
  - [58] Zhiqiang Zuo, Yiyu Zhang, Qiuqiong Pan, Shenming Lu, Yue Li, Linzhang Wang, Xuandong Li, and Guoqing Harry Xu. 2021. Chianina: an evolving graph system for flow-and context-sensitive analyses of million lines of C code. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. 914–929.

Received 2023-02-16; accepted 2023-05-03